https://learnopencv.com/introduction-to-video-classification-and-human-activity-recognition/

Cài đặt

pip install pafy

pip install youtube-dl

pip install moviepy

Cài Open CV

pip install opencv-python

Cài đặt

pip install tensorflow==2.12.0 --upgrade

pip install pydot

pip install pytube

Tải Tệp dữ huấn luyện tại: https://www.crcv.ucf.edu/data/UCF50.rar

Phân loại video bằng Keras

Bước 1: Tải và giải nén tập dữ liệu

Bước 2: Cài đặt các thư viện cần thiết

Bước 3. Khai báo các thư viện cần thiết

Bước 4: Trực quan hóa dữ liệu bằng nhãn của nó

Bước 5: Đọc và xử lý trước tập dữ liệu

Bước 6: Chia dữ liệu thành tập huấn luyện và tập kiểm tra

Bước 7: Xây dựng mô hình

Bước 8: Biên dịch và huấn luyện mô hình

Bước 9: Vẽ đường cong suy hao và độ chính xác của mô hình

Bước 10: Đưa ra dự đoán với mô hình

Bộ dữ liệu chúng tôi đang sử dụng là UCF50 – Bộ dữ liệu nhận dạng hành động.

UCF50 là bộ dữ liệu nhận dạng hành động với 50 danh mục hành động, bao gồm các video chân thực được lấy từ youtube. Tập dữ liệu này là phần mở rộng của tập dữ liệu YouTube Action (UCF11) có 11 danh mục hành động.

Hầu hết các bộ dữ liệu nhận dạng hành động có sẵn đều không thực tế và do các tác nhân dàn dựng. Trong tập dữ liệu của chúng tôi, trọng tâm chính là cung cấp cho cộng đồng thị giác máy tính một bộ dữ liệu nhận dạng hành động bao gồm các video thực tế được lấy từ youtube. Tập dữ liệu của chúng tôi rất khó khăn do có sự thay đổi lớn về chuyển động của máy ảnh, hình dáng và tư thế của đối tượng, tỷ lệ đối tượng, góc nhìn, hậu cảnh lộn xộn, điều kiện chiếu sáng, v.v. Đối với tất cả 50 danh mục, các video được nhóm thành 25 nhóm, trong đó mỗi nhóm bao gồm của hơn 4 clip hành động. Các video clip trong cùng một nhóm có thể có một số đặc điểm chung, chẳng hạn như cùng một người, bối cảnh giống nhau, quan điểm giống nhau, v.v.

50 danh mục hành động của bộ dữ liệu UCF50 được thu thập từ youtube là: Sân bóng chày, Bắn bóng rổ, Bấm ghế, Đạp xe, Đạp xe, Bắn bi-a, Bơi ếch, Sạch và giật, Lặn, Đánh trống, Đấu kiếm, Đánh gôn, Chơi ghi-ta, Nhảy cao, Đua ngựa , Cưỡi ngựa, Hula Hoop, Ném lao, Tung hứng bóng, Nhảy dây, Nhảy Jack, Chèo thuyền kayak, Lunge, Diễu hành quân sự, Trộn bột, Nun chucks, Chơi piano, Ném pizza, Nhảy sào, Ngựa chuôi kiếm, Kéo xà, Đấm, Đẩy Ups, Leo núi trong nhà, Leo dây, Chèo thuyền, Xoay Salsa, Trượt ván, Trượt tuyết, Trượt tuyết, Tung hứng bóng đá, Đánh đu, Chơi Tabla, Thái Cực Quyền, Xích đu quần vợt, Nhảy bạt lò xo, Chơi violin, Đạp xe bóng chuyền, Đi dạo với chó và Yo Yo.

UCF50 là bộ dữ liệu nhận dạng hành động bao gồm:

- 50 Danh mục hành động bao gồm các video YouTube thực tế
- 25 nhóm video cho mỗi danh mục hành động
- Trung bình 133 video cho mỗi danh mục hành động
- 199 Số khung hình trung bình trên mỗi video
- Chiều rộng khung hình trung bình 320 cho mỗi video
- Chiều cao trung bình 240 khung hình trên mỗi video
- 26 khung hình trung bình mỗi giây trên mỗi video

Bước 1: Tải và giải nén tập dữ liệu

+ Tải Tệp dữ huấn luyện tại: https://www.crcv.ucf.edu/data/UCF50.rar

+ Giả nén vào thư mục nào đó. Ví dụ `D:/Python/`

Bước 2: Cài đặt các thư viện cần thiết

- pip install pafy
- pip install youtube-dl
- pip install moviepy
- pip install opencv-python
- pip install tensorflow==2.12.0 --upgrade
- pip install pydot
- pip install pytube

Bước 3. Khai báo các thư viện cần thiết

```
import os
import cv2
import math
import pafy
```

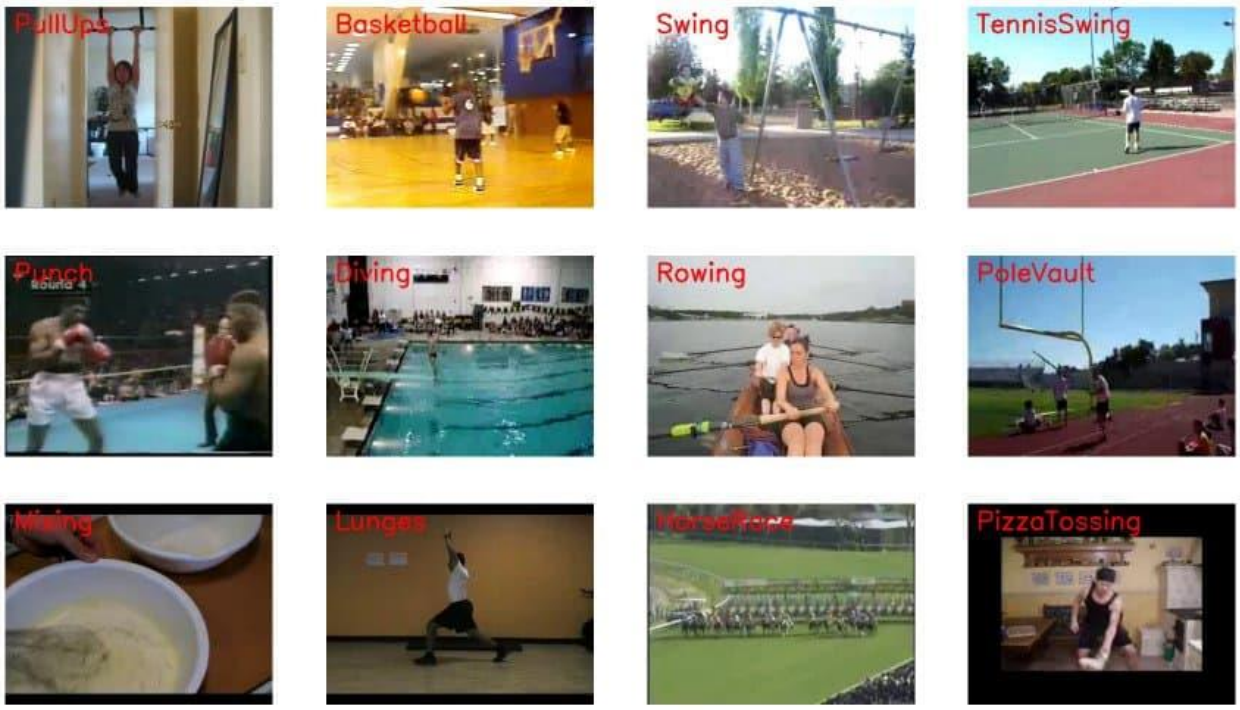
```
import random
import numpy as np
import datetime as dt
import tensorflow as tf
from moviepy.editor import *
from collections import deque
import matplotlib.pyplot as plt
%matplotlib inline

from sklearn.model_selection import train_test_split

from tensorflow.keras.layers import *
from tensorflow.keras.models import Sequential
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.utils import plot_model

seed_constant = 23
np.random.seed(seed_constant)
random.seed(seed_constant)
tf.random.set_seed(seed_constant)
```

Bước 4: Thể hiện dữ liệu cùng nhãn của nó

Chọn một số video ngẫu nhiên từ mỗi loại của tập dữ liệu và hiển thị nó,

```
# Create a Matplotlib figure
plt.figure(figsize = (30, 30))

# Get Names of all classes in UCF50
all_classes_names = os.listdir("D:/Python/UCF50")

# Generate a random sample of images each time the cell runs
random_range = random.sample(range(len(all_classes_names)), 20)

# Iterating through all the random samples
for counter, random_index in enumerate(random_range, 1):

    # Getting Class Name using Random Index
    selected_class_Name = all_classes_names[random_index]

    # Getting a list of all the video files present in a Class Directory
    video_files_names_list =
os.listdir(f"D:/Python/UCF50/{selected_class_Name}")

    # Randomly selecting a video file
    selected_video_file_name = random.choice(video_files_names_list)

    # Reading the Video File Using the Video Capture
    video_reader =
cv2.VideoCapture(f"D:/Python/UCF50/{selected_class_Name}/{selected_video_file
_name}")

    # Reading The First Frame of the Video File
    _, bgr_frame = video_reader.read()
```

```
    # Closing the VideoCapture object and releasing all resources.
    video_reader.release()

    # Converting the BGR Frame to RGB Frame
    rgb_frame = cv2.cvtColor(bgr_frame, cv2.COLOR_BGR2RGB)

    # Adding The Class Name Text on top of the Video Frame.
    cv2.putText(rgb_frame, selected_class_Name, (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2)

    # Assigning the Frame to a specific position of a subplot
    plt.subplot(5, 4, counter)
    plt.imshow(rgb_frame)
    plt.axis('off')
```



Bước 3: Đọc và xử lý tập dữ liệu

- image_height và image_weight: Thay đổi kích thước của tất cả các khung hình trong video về kích thước này.
- max_images_per_class: Số lượng hình ảnh huấn luyện tối đa cho mỗi lớp.
- tập dữ liệu_thư mục: Đường dẫn của thư mục chứa tập dữ liệu được trích xuất.
- class_list: Đây là danh sách các lớp sẽ huấn luyện, chúng ta sẽ huấn luyện 4 lớp sau, bạn có thể thoải mái thay đổi.
    - tai Chi

- Đu đưa
- Đua ngựa
- Đi dạo với một con chó

Lưu ý: Các hằng số image_height, image_weight và max_images_per_class có thể được tăng lên để có kết quả tốt hơn, nhưng hãy lưu ý rằng điều này sẽ tốn kém về mặt tính toán.

```
image_height, image_width = 64, 64
max_images_per_class = 8000

dataset_directory = "D:/Python/UCF50"
classes_list = ["WalkingWithDog", "TaiChi", "Swing", "HorseRace"]

model_output_size = len(classes_list)
```

Trích xuất, thay đổi kích thước và chuẩn hóa khung

Bây giờ chúng ta sẽ tạo một hàm trích xuất khung hình từ mỗi video trong khi thực hiện các thao tác tiền xử lý khác như thay đổi kích thước và chuẩn hóa hình ảnh.

Phương pháp này lấy đường dẫn tệp video làm đầu vào. Sau đó, nó đọc từng khung hình của tệp video, thay đổi kích thước từng khung hình, chuẩn hóa khung đã thay đổi kích thước, nối khung đã chuẩn hóa vào danh sách và cuối cùng trả về danh sách đó.

```
def frames_extraction(video_path):
    # Empty List declared to store video frames
    frames_list = []

    # Reading the Video File Using the VideoCapture
    video_reader = cv2.VideoCapture(video_path)

    # Iterating through Video Frames
    while True:

        # Reading a frame from the video file
        success, frame = video_reader.read()

        # If Video frame was not successfully read then break the loop
        if not success:
            break

        # Resize the Frame to fixed Dimensions
        resized_frame = cv2.resize(frame, (image_height, image_width))
```

```
        # Normalize the resized frame by dividing it with 255 so that each
pixel value then lies between 0 and 1
        normalized_frame = resized_frame / 255

        # Appending the normalized frame into the frames list
        frames_list.append(normalized_frame)

    # Closing the VideoCapture object and releasing all resources.
    video_reader.release()

    # returning the frames list
    return frames_list
```

Tạo tập dữ liệu

Bây giờ chúng ta sẽ tạo một hàm khác gọi là create_dataset(), hàm này sử dụng hàm frame_extraction() ở trên và tạo tập dữ liệu được xử lý

Đây là cách chức năng này hoạt động:

1. Duyệt tất cả các lớp được đề cập trong class_list
2. Với mỗi lớp, duyệt tất cả các tệp video có trong đó.
3. Gọi phương thức frame_extraction trên mỗi tệp video.
4. Thêm các khung được trả về vào danh sách có tên temp_features
5. Sau khi tất cả video của một lớp được xử lý, chọn ngẫu nhiên các khung hình video (bằng max_images_per_class) và thêm chúng vào danh sách gọi là Đặc trưng (features).
6. Thêm nhãn của video đã chọn vào danh sách `nhãn`.
7. Sau khi tất cả video của tất cả các lớp được xử lý, trả về các Đặc trưng và nhãn dưới dạng mảng NumPy.

Vì vậy, khi bạn gọi hàm này, nó sẽ trả về hai danh sách:

- Danh sách các vectơ đặc trưng
- Một danh sách các nhãn liên quan của nó.

```
def create_dataset():

    # Declaring Empty Lists to store the features and labels values.
    temp_features = []
    features = []
    labels = []

    # Iterating through all the classes mentioned in the classes list
    for class_index, class_name in enumerate(classes_list):
        print(f'Extracting Data of Class: {class_name}')
```

```
        # Getting the list of video files present in the specific class name
directory
        files_list = os.listdir(os.path.join(dataset_directory, class_name))

        # Iterating through all the files present in the files list
        for file_name in files_list:

            # Construct the complete video path
            video_file_path = os.path.join(dataset_directory, class_name,
file_name)

            # Calling the frame_extraction method for every video file path
            frames = frames_extraction(video_file_path)

            # Appending the frames to a temporary list.
            temp_features.extend(frames)

        # Adding randomly selected frames to the features list
        features.extend(random.sample(temp_features, max_images_per_class))

        # Adding Fixed number of labels to the labels list
        labels.extend([class_index] * max_images_per_class)

        # Emptying the temp_features list so it can be reused to store all
frames of the next class.
        temp_features.clear()

    # Converting the features and labels lists to numpy arrays
    features = np.asarray(features)
    labels = np.array(labels)

    return features, labels
```

Gọi phương thức create_dataset để trả về các Đặc trưng và nhãn.

```
features, labels = create_dataset()
```

Kết quả:

```
Extracting Data of Class: WalkingWithDog
Extracting Data of Class: TaiChi
Extracting Data of Class: Swing
Extracting Data of Class: HorseRace
```

Chuyển đổi nhãn lớp thành một vectơ được mã hóa theo phương pháp one hot encoded.

```
# Using Keras's to_categorical method to convert labels into one-hot-encoded
vectors
one_hot_encoded_labels = to_categorical(labels)
```

Bước 4: Chia dữ liệu thành tập huấn luyện và tập kiểm tra

Bây giờ chúng ta có hai mảng, một mảng chứa tất cả hình ảnh. Cái thứ hai chứa tất cả các nhãn lớp ở một định dạng được mã hóa. Chúng ta sẽ phân chia dữ liệu của mình để tạo tập huấn luyện và tập kiểm tra. Chúng ta phải xáo trộn dữ liệu trước khi phân chia.

```
features_train, features_test, labels_train, labels_test =
train_test_split(features, one_hot_encoded_labels, test_size = 0.2, shuffle =
True, random_state = seed_constant)
```

Bước 5: Xây dựng mô hình

Bây giờ là lúc tạo mô hình CNN, ta sẽ tạo một mô hình Phân loại CNN đơn giản với hai lớp CNN.

```
# Let's create a function that will construct our model
def create_model():

    # We will use a Sequential model for model construction
    model = Sequential()

    # Defining The Model Architecture
    model.add(Conv2D(filters = 64, kernel_size = (3, 3), activation = 'relu',
input_shape = (image_height, image_width, 3)))
    model.add(Conv2D(filters = 64, kernel_size = (3, 3), activation =
'relu'))
    model.add(BatchNormalization())
    model.add(MaxPooling2D(pool_size = (2, 2)))
    model.add(GlobalAveragePooling2D())
    model.add(Dense(256, activation = 'relu'))
    model.add(BatchNormalization())
    model.add(Dense(model_output_size, activation = 'softmax'))

    # Printing the models summary
    model.summary()

    return model


# Calling the create_model method
model = create_model()

print("Model Created Successfully!")
```

Kết quả

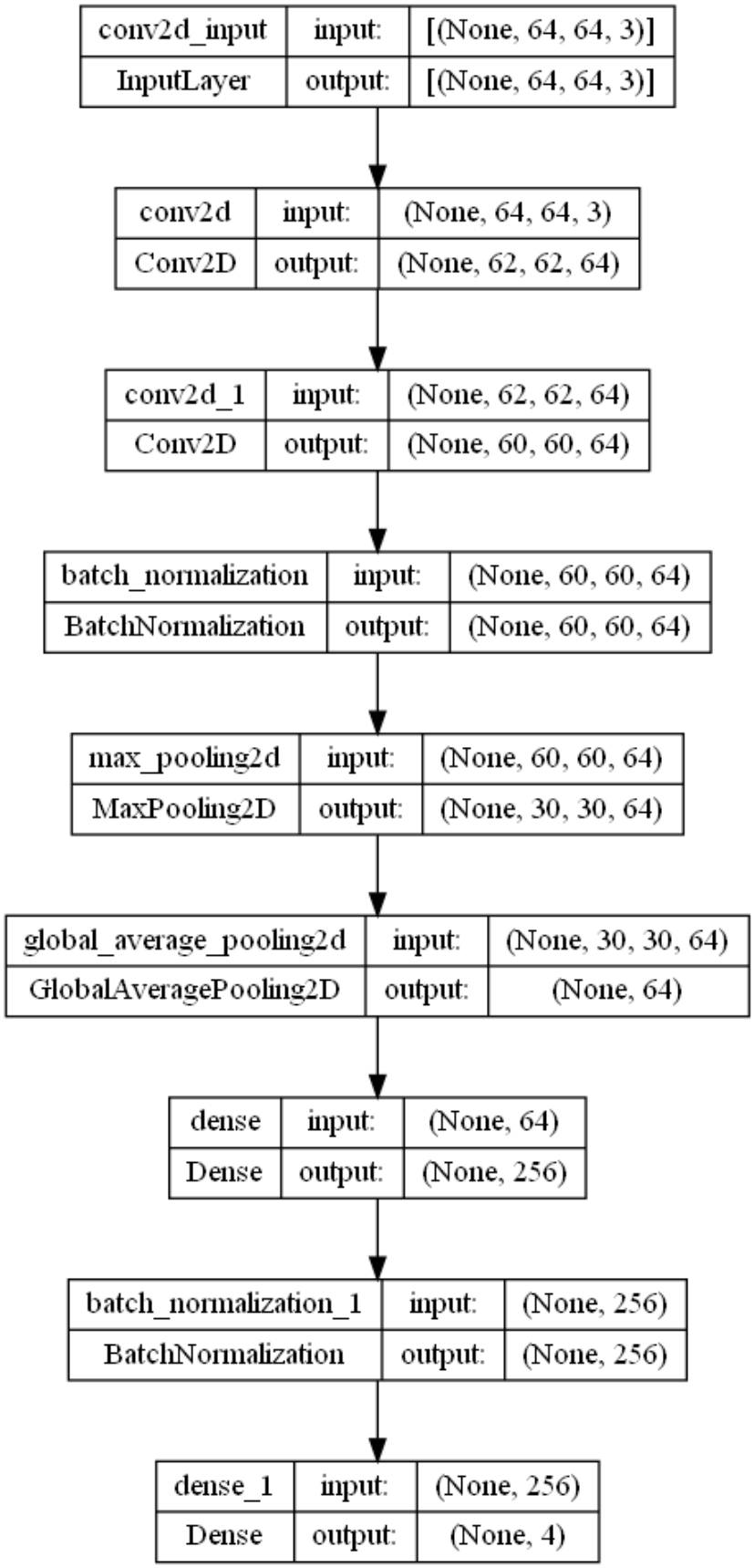
```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 62, 62, 64)        1792
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 60, 60, 64)        36928
_________________________________________________________________
batch_normalization (BatchNo (None, 60, 60, 64)        256
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 30, 30, 64)        0
_________________________________________________________________
global_average_pooling2d (Gl (None, 64)                0
_________________________________________________________________
dense (Dense)                (None, 256)               16640
_________________________________________________________________
batch_normalization_1 (Batch (None, 256)               1024
_________________________________________________________________
dense_1 (Dense)              (None, 4)                 1028
=================================================================
Total params: 57,668
Trainable params: 57,028
Non-trainable params: 640
_________________________________________________________________
```

Kiểm tra cấu trúc của mô hình:

Sử dụng hàm story_model, chúng ta có thể kiểm tra cấu trúc của mô hình cuối cùng. Điều này thực sự hữu ích khi chúng ta tạo một mạng phức tạp và bạn muốn đảm bảo rằng chúng ta đã xây dựng mạng một cách chính xác.

```
plot_model(model, to_file = 'model_structure_plot.png', show_shapes = True,
show_layer_names = True)
```

Kết quả:


conv2d_input
input:
[(None, 64, 64, 3)]
InputLayer
output:
[(None, 64, 64, 3)]
conv2d
input:
(None, 64, 64, 3)
Conv2D
output:
(None, 62, 62, 64)
conv2d_1
input:
(None, 62, 62, 64)
Conv2D
output:
(None, 60, 60, 64)
batch_normalization
input:
(None, 60, 60, 64)
BatchNormalization
output:
(None, 60, 60, 64)
max_pooling2d
input:
(None, 60, 60, 64)
MaxPooling2D
output:
(None, 30, 30, 64)
global_average_pooling2d
input:
(None, 30, 30, 64)
GlobalAveragePooling2D
output:
(None, 64)
dense
input:
(None, 64)
Dense
output:
(None, 256)
batch_normalization_1
input:
(None, 256)
BatchNormalization
output:
(None, 256)
dense_1
input:
(None, 256)
Dense
output:
(None, 4)